

In-class live session · part two

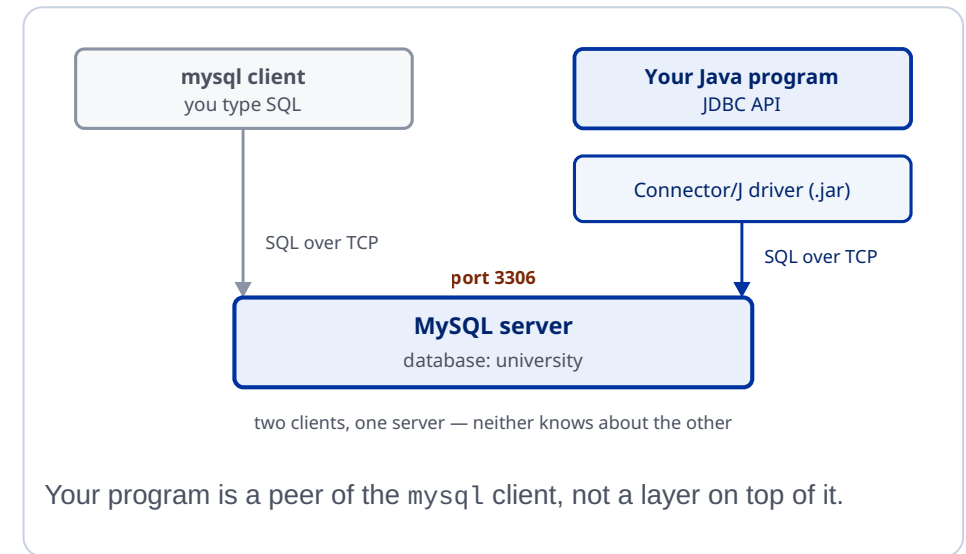
Java to the database

Same machine, same server, same UNIVERSITY database. The only thing that changes is who is asking — a Java program instead of you at a prompt.

Everything we ran this morning went through the `mysql` client. That client is just a program that opens a socket, sends SQL, and formats what comes back. Now we write one of those ourselves, and discover that the gap between a result set and a Java variable is wider than it looks.

You already have the database

Nothing new to create. If you ran `university_bootstrap.sql`, the four tables and their rows are sitting there waiting. We are only adding a second way to reach them.



Why this is its own topic

What actually changes

The SQL does not change at all. What changes is that the answer now has to become Java objects — and the two languages disagree about almost everything.

The **impedance mismatch**, concretely.

SQL thinks in	Java thinks in
whole sets at once	one object at a time, in a loop
rows with named columns	fields, objects, getters
NULL as a third truth value	null refs, and primitives that cannot be null
DECIMAL(3,2), CHAR(5)	double, String, BigDecimal
errors as numbered codes	exceptions you must catch

What JDBC is

A standard Java API — interfaces like `Connection`, `Statement`, `ResultSet` — plus a vendor-written **driver** that implements them for one database.

Your code is written against the interfaces. Swapping MySQL for PostgreSQL means a different jar and a different URL, not different code. That is the entire promise, and it mostly holds.

Two places this bites today

A `ResultSet` is a cursor, not a list — you walk it once, forward.

And SQL NULL read into a Java primitive does not throw, does not warn, and does not give you null. It gives you zero. We will watch that happen.

What has to be installed

Three things, and you already have two of them.

Everything needed to run Java against the database on this machine.

Component	What to install	How to check	Status
MySQL server	Done this morning — 9.7 LTS, running on port 3306	<code>mysql -u root -p</code>	already have it
JDK	A current LTS: JDK 21 or JDK 25 . You need the <i>JDK</i> , not just a JRE — you are compiling.	<code>javac -version</code> <code>java -version</code>	assumed installed
Connector/J	MySQL's JDBC driver, as a single .jar. This is the one you are missing. Version 9.7.0 to match the 9.7 server.	the jar file exists where you put it	install this

Why both numbers are 9.7

Connector/J now tracks the server's version scheme, so 9.7.0 is the driver that pairs with MySQL 9.7 LTS. There is also a 26.x innovation line, same calendar-versioning trap as the server. Take 9.7.0.

The driver is not fussy about Java

Connector/J only requires JDK 8 or newer, so your JDK choice is about this course, not about the driver. If `javac -version` prints anything from 17 upward you are fine.

Getting the driver

- 1 Go to `dev.mysql.com/downloads/connector/j/`
- 2 For "Select Operating System", choose **Platform Independent**. The driver is pure Java; there is no Windows build and no Mac build.
- 3 Download the ZIP or TAR archive and unpack it.
- 4 Inside is one file that matters:
`mysql-connector-j-9.7.0.jar`
- 5 Put that jar somewhere stable and easy to type — a `lib` folder next to your source is ideal.

```
myproject/  
  JdbcLab.java  
  lib/  
    mysql-connector-j-9.7.0.jar
```

"Platform Independent" is not a trick question

Connector/J is a **Type 4** driver: pure Java, speaking the MySQL network protocol directly. It does not wrap any native library, which is why one jar works on every operating system. If you are being offered an MSI, you are on the wrong download page.

If you use Maven or Gradle

Declare the dependency instead of downloading anything, and the build tool handles the classpath for you:

```
<!-- Maven -->  
<dependency>  
  <groupId>com.mysql</groupId>  
  <artifactId>mysql-connector-j</artifactId>  
  <version>9.7.0</version>  
</dependency>
```

The classpath

This is where this lab goes wrong for most people, and it is not a Java concept problem — it is a "where is the jar" problem.

macOS and Linux — separator is a colon:

```
$ javac -cp lib/mysql-connector-j-9.7.0.jar JdbcLab.java
$ java -cp .:lib/mysql-connector-j-9.7.0.jar JdbcLab
```

Windows — separator is a semicolon:

```
> javac -cp lib\mysql-connector-j-9.7.0.jar JdbcLab.java
> java -cp .;lib\mysql-connector-j-9.7.0.jar JdbcLab
```

Do not lose the dot

The `.` in `.:lib/...` is your own compiled class. Leave it out and Java finds the driver but not your program: Could not find or load main class JdbcLab.

Compile time vs run time

`javac` technically does not need the driver at all — your code only mentions `java.sql` interfaces. But `java` absolutely does, because the driver has to be found and loaded. Passing it to both is the habit that avoids the confusing half-working state.

In an IDE

There is no command line to get right — you add the jar to the project's libraries or build path instead. **IntelliJ**: File → Project Structure → Libraries → +. **Eclipse**: right-click project → Build Path → Add External Archives. **VS Code**: add it under Java Project → Referenced Libraries.

The error you will get if you skip this

```
java.sql.SQLException
No suitable driver found for
jdbc:mysql://localhost:3306/university
```

The connection URL

One string carries the protocol, the machine, the port and the database. Get it wrong and the error you get usually names which part.

jdbc:mysql://localhost:3306/university

which driver	which machine	which port	which database
always this, for MySQL	this computer	MySQL's default	what USE selected this morning

On this machine, only the last part is a decision; the rest is boilerplate.

If the connection is refused over security settings

A fresh MySQL uses strong password encryption, and over a plain local connection the driver sometimes needs permission to fetch the server's public key. If you hit that, append `?allowPublicKeyRetrieval=true&useSSL=false` to the URL. That is acceptable for a local teaching database on your own machine and **not** acceptable for anything real.

The smallest useful program

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class TestConnection {
    static final String URL =
        "jdbc:mysql://localhost:3306/university";
    static final String USER = "root";
    static final String PASS = "your-password";

    public static void main(String[] args) {
        try (Connection conn =
            DriverManager.getConnection(URL, USER, PASS)) {
            System.out.println("Connected to: " + conn.getCatalog());
        } catch (SQLException e) {
            System.err.println("Failed: " + e.getMessage());
            System.err.println("SQLState: " + e.getSQLState());
        }
    }
}
```

Three things worth noticing

No `Class.forName`. Older tutorials open with `Class.forName("com.mysql.cj.jdbc.Driver")`. Since JDBC 4.0 the driver registers itself just by being on the classpath. If you see that line, the tutorial predates 2007.

The `try (...)` is doing work. That is try-with-resources: the connection is closed when the block ends, including when an exception is thrown. A leaked connection stays open on the server until it times out.

SQLState is the useful part of a failure. It is a standard five-character code. 08001 means the driver could not be found; 28000 means your password was rejected.

About that password in the source

Fine for a lab on your own machine, wrong everywhere else — it ends up in version control. Real programs read credentials from a config file or an environment variable.

The shape of all of them

Every JDBC program you will ever write is these five steps. Once you can see them, the API stops feeling large.

- 1 **Get a Connection** — one TCP session to the server.
- 2 **Make a Statement or PreparedStatement** — the thing that carries SQL.
- 3 **Execute.** `executeQuery` for a SELECT, which hands back a `ResultSet`; `executeUpdate` for INSERT, UPDATE and DELETE, which hands back a row count.
- 4 **Walk the ResultSet** — a cursor, one row at a time, forward only by default.
- 5 **Close everything**, in reverse order — or let try-with-resources do it, which is why we nest the declarations.

Which execute method

```
ResultSet rs = st.executeQuery(sql);    // SELECT
int      n  = st.executeUpdate(sql);    // INSERT/
                                           // UPDATE/
                                           // DELETE/DDL
```

Closing is not optional

Connections are a limited server resource. A program that opens one per query and never closes them will run fine on your laptop for about four minutes and then start collecting Too many connections. Try-with-resources makes this a non-issue, which is why it is the only form shown in this deck.

Walking a result set

```
String sql =
    "SELECT StudentID, Name, Major, GPA " +
    "FROM STUDENT ORDER BY StudentID";

try (Connection conn = DriverManager.getConnection(URL, USER,
    PASS);
     Statement st = conn.createStatement();
     ResultSet rs = st.executeQuery(sql)) {

    while (rs.next()) {
        String id = rs.getString("StudentID");
        String name = rs.getString("Name");
        String major = rs.getString("Major");
        double gpa = rs.getDouble("GPA");
        System.out.printf("%-6s %-14s %-5s %.2f%n",
                           id, name, major, gpa);
    }
}
```

rs.next() does two jobs

It moves the cursor forward *and* reports whether there was anywhere to move to. A fresh ResultSet sits *before* the first row, which is why the loop is while (rs.next()) and not a do-while. Forget the call and you get an exception, not an empty loop.

Columns by name or by number

rs.getString("Name") and rs.getString(2) both work. Indexes start at **1**, not 0 — JDBC predates the convention you are used to. Use names; they survive someone editing the SELECT list.

One pass, forward

By default you cannot go back, and you cannot read it twice. If you need the rows again, copy them into a List as you go.

Step the cursor yourself

Press `rs.next()` and watch what the cursor does and what the getters return at each position. Note where it starts, and what the last call gives back.

The result set is open and the cursor sits before the first row. No getter may be called yet. 8 rows are waiting.

The result set, with the cursor position marked.

Cursor	StudentID	Name	GPA
	S1001	Ana Reyes	3.72
	S1002	Ben Okafor	3.15
	S1003	Chloe Nguyen	3.95
	S1004	Diego Santos	2.88
	S1005	Emma Larsen	3.40
	S1006	Farid Haddad	3.61
	S1007	Grace Kim	3.33
	S1008	Henry Patel	NULL

```
position : before first row
rs.next(): not called yet
```

```
rs.getString("Name") -> SQLException
rs.getDouble("GPA") -> SQLException
```

Watch for two moments

At **row 8**, `getDouble("GPA")` returns `0.0` even though the stored value is `NULL`. Henry has no GPA, and Java has just told you he has a zero.

At the **ninth call**, `next()` returns `false` and the loop ends. That `false` is the only thing that stops you.

Henry Patel has a 0.00 GPA

He does not, of course. But that is what the program on the previous slide prints, and nothing anywhere warns you.

What that code actually printed:

S1001	Ana Reyes	CS	3.72
S1002	Ben Okafor	CS	3.15
S1003	Chloe Nguyen	MATH	3.95
S1004	Diego Santos	CS	2.88
S1005	Emma Larsen	BIOL	3.40
S1006	Farid Haddad	MATH	3.61
S1007	Grace Kim	CS	3.33
S1008	Henry Patel	null	0.00

Why this happens

`getDouble` returns a primitive `double`, and a primitive cannot be null. The JDBC specification says to return **0** for SQL NULL. Same for `getInt`: you get 0. It is behaving exactly as documented, and it is still a wrong answer in your report.

The fix — `wasNull()`, asked straight after the get:

```
String major = rs.getString("Major");
if (rs.wasNull()) major = "(undeclared)";

double gpa    = rs.getDouble("GPA");
String gpaOut = rs.wasNull()
    ? "  --  "
    : String.format("%.2f", gpa);
```

S1007	Grace Kim	CS	3.33
S1008	Henry Patel	(undeclared)	--

The rule

`wasNull()` reports on the *most recent* getter call, so it must come immediately after. For object types like `getString` you can just test for null instead — the trap is only with primitives. Or select `getObject(..., Double.class)` and get a real null back.

Values are not SQL

The moment a query depends on something a user typed, stop building strings.
Put a ? in the SQL and hand the value over separately.

Gluing strings — don't:

```
String sql =
    "SELECT Name FROM STUDENT WHERE Major = '"
    + major + "'";

Statement st = conn.createStatement();
ResultSet rs = st.executeQuery(sql);
```

Parameterized — do:

```
String sql =
    "SELECT Name FROM STUDENT WHERE Major = ?";

try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, major);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) { ... }
    }
}
```

What the ? actually does

The SQL text and the values travel to the server **separately**. The server parses the statement once, with the ? as a hole, and then fills the hole with a value that is never parsed as SQL. A quote inside the value is just a character. There is nothing to escape because nothing is being assembled.

Parameters are 1-indexed too

setString(1, ...) is the first ?. There is a typed setter for each kind — setInt, setDouble, setNull(n, Types.CHAR) for a deliberate NULL.

It is also faster

A prepared statement reused in a loop is parsed once and executed many times. The security argument is the one that matters, but you are not paying for it.

The reason it matters

Let's break it on purpose

Same program, same database. The only thing that changes is what the user types into the "major" box.

1 · Honest input: CS

```
SQL sent: SELECT Name, Major FROM STUDENT
          WHERE Major = 'CS'
Ana Reyes
Ben Okafor
Diego Santos
Grace Kim
-> 4 row(s)
```

2 · Crafted input: ' OR '1'='1

```
SQL sent: SELECT Name, Major FROM STUDENT
          WHERE Major = '' OR '1'='1'
Ana Reyes      Ben Okafor
Chloe Nguyen   Diego Santos
Emma Larsen    Farid Haddad
Grace Kim      Henry Patel
-> 8 row(s)
```

3 · Same crafted input, through a PreparedStatement:

```
SQL sent: SELECT Name, Major FROM STUDENT
          WHERE Major = ?
          [param = ' OR '1'='1]
-> 0 row(s)
```

Read line 2 again

The user's quote *closed* the string the programmer opened, and everything after it became part of the query. '1'='1' is true for every row, so the WHERE clause stopped filtering. The whole table walked out — including Henry, who has no major at all.

And why the third one is safe

It searched for students whose major is literally the eleven-character string ' OR '1'='1. There are none, so it returned nothing. That is the correct answer to a nonsense question — which is precisely what you want.

A SELECT is the gentle version. The same hole with executeUpdate takes DELETE statements.

Changing data

Writing, and being refused

```
String sql = "INSERT INTO ENROLLED_IN " +
    "(StudentID, CourseID, Grade) VALUES (?, ?, ?)";

try (Connection conn = DriverManager.getConnection(URL, USER,
    PASS);
    PreparedStatement ps = conn.prepareStatement(sql)) {

    ps.setString(1, "S1008");
    ps.setString(2, "CS410");
    ps.setNull(3, Types.CHAR);    // in progress

    int rows = ps.executeUpdate();
    System.out.println("Inserted " + rows + " row(s).");

    ps.executeUpdate();          // run it a second time

} catch (SQLException e) {
    System.out.println("Refused : " + e.getMessage());
    System.out.println("SQLState: " + e.getSQLState()
        + " code: " + e.getErrorCode());
}
```

Inserted 1 row(s).

Refused : Duplicate entry 'S1008-CS410'
for key 'PRIMARY'
SQLState: 23000 code: 1062

You have seen this error before

That is the same **1062** that refused you at the `mysql>` prompt this morning, printing the same hyphenated pair. The constraint does not care who is asking. It arrives in Java as a `SQLException` you can catch and act on — which is the point of putting the rule in the database rather than in every program that touches it.

Autocommit is on by default

Each statement commits as it runs. For several writes that must succeed or fail together, turn it off and take control — `conn.setAutoCommit(false)`, then `conn.commit()` or `conn.rollback()`. That is Unit 8, and the reason transactions get three lectures.

Reference

When it does not work

Read the `SQLState` and the vendor code before you read anything else. They identify the problem precisely; the message text varies between driver versions.

The five failures that account for nearly every stuck student.

SQLState	Code	What it says	What is actually wrong
08001	–	No suitable driver found	The jar is not on the run-time classpath. This is a -cp problem, not a code problem.
08S01	–	Communications link failure / connection refused	Nothing is listening. The server is stopped, or the port in the URL is wrong.
28000	1045	Access denied for user ... (using password: YES)	Wrong username or password. The server was reached, so the URL is fine.
42000	1049	Unknown database 'universty'	Typo in the database name at the end of the URL.
42000	1044	Access denied for user ... to database ...	The database exists but this user has no rights on it. A grant problem, not a password problem.

Reading an exception properly

Three calls tell you nearly everything: `e.getSQLState()` for the standard class of failure, `e.getErrorCode()` for MySQL's own number, and `e.getMessage()` for the human sentence. Print all three while you are learning; a stack trace alone usually hides the one line that matters.

Wrap-up

Closing notes

What today added

- **A driver is just a jar on the classpath.** Most first-time failures are `-cp`, not code.
- **Five steps, always:** connect, prepare, execute, walk the results, close.
- **A `ResultSet` is a cursor**, starting before row one, going forward once.
- **SQL NULL read into a primitive is zero.** `wasNull()` is the only thing standing between you and a wrong number.
- **Never concatenate a value into SQL.** One quote in the wrong place emptied the table in front of you.
- **Constraints follow you.** The same 1062 refuses the prompt and the program.

Try these before next class

1. Change the query to the CS410 roster — a join across `STUDENT` and `ENROLLED_IN` — and print it.
2. Make it take the course ID as a command-line argument, through a `PreparedStatement`.
3. Then try to break your own program with `' OR '1'='1` and satisfy yourself that you cannot.
4. Delete the jar from your `-cp` and read the error, so you recognize it instantly later.

Where this is going

Unit 5 continues with stored procedures and with the deeper version of the injection story. The Final Project needs an application talking to your schema, and this is that connection — the code on these slides is close to what you will start from.

Reference

Resources

Downloads

- **Connector/J** — dev.mysql.com/downloads/connector/j/
Choose "Platform Independent". Version 9.7.0 pairs with MySQL 9.7 LTS.
- **A JDK**, if you need one — adoptium.net
Free builds of JDK 21 and 25, both LTS. Take the JDK, not the JRE.

Documentation

- **Connector/J Developer Guide** — dev.mysql.com/doc/connector-j/en/
- **Oracle's JDBC tutorial** — docs.oracle.com/javase/tutorial/jdbc/
Dated in places, but the best free walkthrough of the API itself.

Course materials

- `university_bootstrap.sql` — rebuilds the database these programs query.
- Elmasri & Navathe, **Ch. 10** — database access from applications.

The commands, in one place

```
$ javac -cp lib/mysql-connector-j-9.7.0.jar \  
    JdbcLab.java  
$ java -cp .:lib/mysql-connector-j-9.7.0.jar \  
    JdbcLab
```

On Windows, swap : for ; and / for \.

The URL, for copying

```
jdbc:mysql://localhost:3306/university
```

Add `?allowPublicKeyRetrieval=true&useSSL=false` only if the connection is refused over key retrieval, and only locally.

One question to take away

Your program asked for eight students and got eight rows. Where did those rows live between `executeQuery` and your loop reading them — on the server, or in your program's memory? The answer decides what happens when the table has ten million rows.